

本科生课程论文成绩评定表

教师填写	2024——2025 学年 第 2 学期 课程名称 大数据案例分析与实践 要求提交论文的日期 2025 年 06 月 11 日	课程类别： 必修[] 选修[√] 考试方式： 课程论文
学生填写	经济学院 学院 经济统计学 专业 2023 级 姓名 叶伟丰 学号 2023102051 内招[√] 外招[] 论文题目 基于自然语言处理电影口碑分析：《复仇者联盟 4》与《雷霆特工队》影评对比	
任课教师意见	论文评语：	
	评定分数： 签章： 年 月 日	
参考评价指标： 1. 选题是否结合实际，是否有创新性； 2. 立论是否鲜明，观点是否正确，是否反映培养的目标要求； 3. 文献收集的广泛程度，是否了解本课题的研究状况及最新的进展情况； 4. 是否较系统地掌握了本课程的基础知识和专业知识，是否理论联系实际； 5. 是否结构合理、层次分明，文字和图表是否规范，学风是否严谨		

基于自然语言处理的电影口碑分析：《复仇者联盟 4》与《雷霆特工队》影评对比

摘要

本研究对比分析了漫威电影宇宙（MCU）作品《复仇者联盟 4：终局之战》（以下简称“复联 4”）与《雷霆特工队》（以下简称“雷特”）的观众口碑差异，旨在探讨 MCU 在阶段演变中的成功要素与叙事挑战。研究通过网络爬虫获取两部影片的豆瓣中文影评数据，进行清洗、分词等预处理后，采用词频分析、词云图可视化、主题建模（LDA）和情感分析等方法，对比两部影片影评中的高频词汇、隐含主题和情感倾向。此外，鉴于豆瓣网的显示机制无法展示近期真实好评率，研究还构建了基于 LSTM 和 RNN 神经网络模型的文本分类器，将“好评”与“差评”作为训练集，对“一般”评论进行二分类，计算出好评率，以量化两部影片的口碑差异。词频分析和词云图显示，《复联 4》的影评关键词集中于“漫威”、“复仇者”、“英雄”等，情感词汇高频出现，表明观众对该片的史诗级英雄叙事和情感共鸣高度认可；而《雷霆特工队》的影评则更关注“角色”和“故事”，并突出“反派”、“黑暗”等主题，情感强度相对较弱。主题建模结果也印证了这一差异。情感分析结果显示，《复联 4》的正面评论比例略高于《雷霆特工队》。基于 LSTM 和 RNN 的文本分类结果表明，《复联 4》的好评率显著高于《雷霆特工队》。综上，数据分析结果表明，《复联 4》在观众口碑方面显著优于《雷霆特工队》，其成功在于深厚的情感积淀和史诗闭环，而《雷霆特工队》则因情感厚度不足和叙事短板未能获得同等认可。本研究为理解 MCU 阶段演变中的成功要素和面临的叙事挑战提供了量化分析和经验性证据。

关键词：观众口碑 影评分析 自然语言处理 基于 LSTM 和 RNN 的文本分类

一、研究背景及问题描述

漫威电影宇宙 (MCU) 在其巅峰作品《复仇者联盟 4: 终局之战》(Avengers: Endgame, 2019) 之后, 面临持续性的口碑挑战。据广泛认可的影评聚合平台烂番茄 (Rotten Tomatoes) 数据显示, 《复仇者联盟 4: 终局之战》(下称复联 4) 获得了极高的评价, 其“Tomatometer”(认证影评人评分) 新鲜度高达 94%(基于 348 条评论, 数据截至 2024 年 7 月), 并在另一重要指标平台 Metacritic 上获得 78/100(基于 58 家主流媒体评论) 的“普遍赞誉”评级。然而, 漫威电影后续多部作品如《雷神 4: 爱与雷霆》(Thor: Love and Thunder, 2022) (烂番茄新鲜度 64% / Metascore 57)、《蚁人与黄蜂女: 量子狂潮》(Ant-Man and the Wasp: Quantumania, 2023) (烂番茄新鲜度 46% / Metascore 48)、《惊奇队长 2》(The Marvels, 2023) (烂番茄新鲜度 62% / Metascore 50) 等, 在评论界反响相对平庸或欠佳, 整体呈现口碑下滑趋势。

近期上映的《雷霆特工队》(Thunderbolts, 2024) 被视为漫威影业试图扭转颓势的一部作品, 并获得了相对积极的初始评价。其在烂番茄的 Tomatometer 新鲜度为 78%(基于 218 条评论, 数据截至 2024 年 7 月), Metascore 评分为 57/100(基于 41 家媒体评论)。虽然该片口碑显著优于复联 4 后的部分作品(如《量子狂潮》与《惊奇队长 2》), 但距离复联 4 的历史性高口碑仍存在明显差距(Tomatometer 差距约 16 个百分点 / Metascore 差距 21 分)。

鉴于复联 4 与《雷霆特工队》(下称雷特) 均涉及多元英雄集结的叙事模式, 但在叙事结构、角色塑造、主题深度及评论接受度上存在显著差异, 本研究旨在运用多项自然语言处理技术对两部电影的公开中文影评进行系统性对比。核心目标在于: 通过分析文本中的常见词语和主题分类、影评人情感、对一般评论计算其中好评率, 揭示两部影片在观众口碑上的差异; 并尝试解析雷特相较于复联 4 的优劣所在及其口碑差距的具体表现形式, 为理解 MCU 阶段演变中的成功要素和面临的叙事挑战提供经验性证据支持。

二、数据描述

2.1 数据获取

本研究所分析的影评文本均采集自中国大陆领先的影视资讯与评论平台——豆瓣网。通过网络爬虫脚分别爬取复联 4 与雷特两部电影的好评、一般评论、差评各 400 条(豆瓣网只显示 400 条数据), 两部电影合计 2400 条影评数据。

2.2 数据清洗与分词

在进行进一步的分析前, 需要对所爬取的影评进行文本数据的清洗。除进行常规的标点符号和换行符的处理外, 研究观察发现影评中有较多的 Unicode 表情符号, 需要进行着重处理。此外评论中含有部分英文字符, 此处只希望对中文文本进行分析, 故将所有的英文字符做删去处理。再完成上述字符删除后, 还需将清洗后的文本做去空处理, 防止部分文本因删去上述符号后变为空评论影响后续的分析。

对于中文分词问题, 常采用 Jieba 库进行处理。此处分词词典采用搜狗词库中的复仇者联盟词库, 并手动添加雷特中的专有名词以及部分词典当中并未涉及的缩写; 停用词词库采用哈工大停用词词库, 并根据实际分词情况对该词库手动增加部分新停用词。

上述处理写为一个类存储在一个 python 文件中方便后续调用。

三、模型建立

3.1 词频分析与词云图绘制



图1 两部电影好评词云图对比

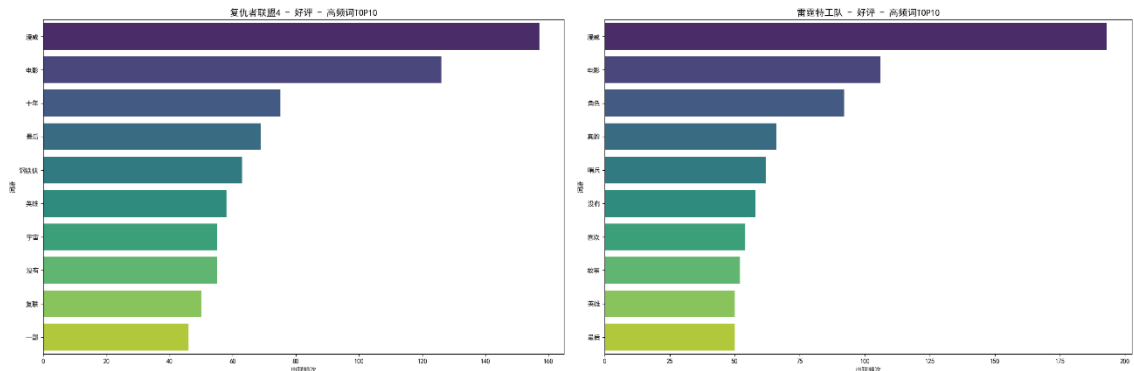


图2 两部电影好评高频词对比

通过对复联4和雷特的好评词云分析可见，两部漫威作品在观众关注点和情感共鸣上呈现显著差异。复联4的核心词汇强烈聚焦于“漫威”、“复仇者”、“英雄”、“宇宙”，彰显了观众对其作为宏大的漫威宇宙集结篇及史诗级英雄叙事的深深认可。“感谢”、“情感”等高频情感词则印证了电影在调动观众情绪上的巨大成功，形成了强烈的情感共鸣。

“最后”、“结局”、“十年”，表明观众高度肯定该片作为阶段性史诗篇章的总结地位。同时，核心角色“钢铁侠”、“美队”的突出体现了观众对关键人物命运走向的高度关注。

相较之下，《雷震特工队》的观众反馈呈现不同面貌。虽然“漫威”、“电影”、“英雄”仍然位列关键词，但其显著性相较《复联4》有所减弱。观众兴趣更明显地转向了“角色”与“故事”，尤其体现在对如“叶莲娜”、“冬兵”等个体角色的讨论上，并且凸显了“反派”、“黑暗”的主题——反映出观众对其聚焦非传统英雄角色定位的认知。评价性词汇如“不错”、“有意思”情感并不太强烈，表明观众整体评价是积极的，但可能缺乏特别的亮点。整体而言，其词云体现了对漫威品牌元素的持续认可，同时更聚焦于角色塑造本身、故事叙述以及反英雄题材的探索，观众观感偏向正向，但情感深度和轰动效应远不及《复联4》。



图3 两部电影一般评论词云图对比

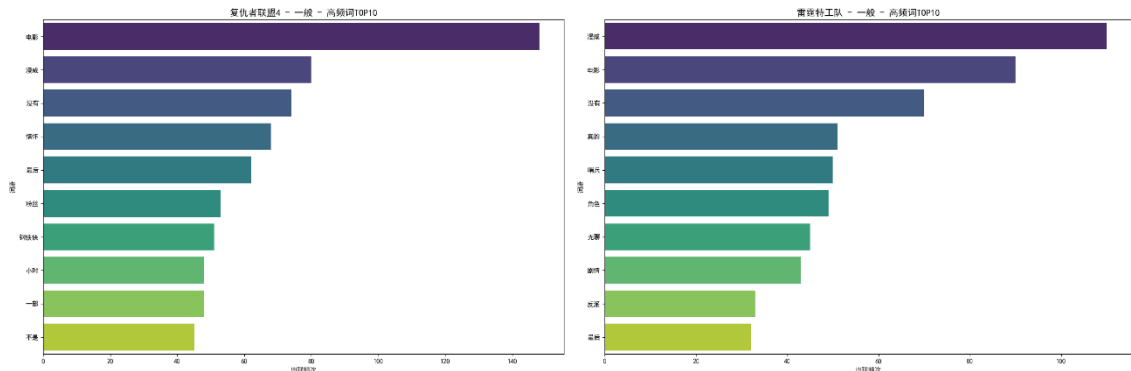


图4 两部电影一般评论高频词对比

通过词云及高频词对比，复联4与雷特的观众反馈差异显著。复联4的核心是“情怀”、“最后”与“钢铁侠”，凸显其作为漫威十年史诗终章的强大情感驱动力，高频词“没有”可能反映了部分观众对结局的不满，但“感动”、“怀旧”等词印证了整体强烈共鸣。反之，雷特聚焦“反派/反英雄”、“动作”及“剧情”，其关键词“角色”与“真的”暗示观众更关注角色塑造真实性与故事逻辑，评价词汇趋于中性（如“好看”、“有趣”），但“没有”高频且“失望”等负面词共存，显示口碑更分化，情感冲击明显弱于前者。复联4借深厚情感积淀与史诗闭环收获普遍赞誉，争议难掩里程碑地位；而雷特虽有反英雄题材创新，但因情感厚度不足及叙事（剧情、节奏）短板，未能获得同等深度认可，凸显了在漫威品牌加持下，情感共鸣的强度仍是决定评价高度的核心因素。



图5 两部电影差评词云图对比

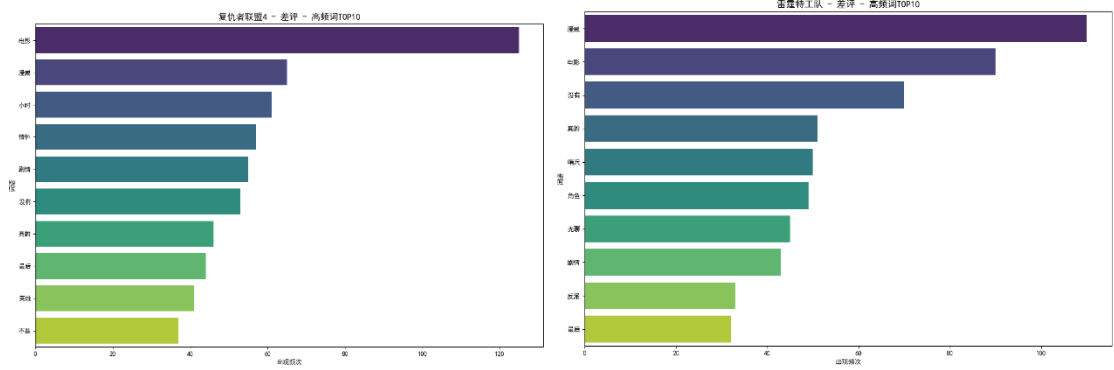


图6 两部电影差评高频词对比

根据上面图像分析显示，两部漫威电影的负面焦点呈现显著差异。复联4的差评核心围绕“剧情逻辑”、“过度情感渲染”及“消费情怀”展开，高频词“小时”（指电影冗长）、“结局”和“情怀”直指观众对史诗终章的高期待落空——既批评其用煽情掩盖叙事缺陷，亦质疑角色命运处理的合理性。反观雷神，差评则集中于“剧情”、“无聊”等制作硬伤，“烂片”、“尴尬”等词频映证部分观众的不满情绪，高频词“角色”凸显观众对人物深度的失望。

复联4作为重磅收官作，其差评暴露了情怀叙事与严密剧情难以兼得的矛盾；而雷特的争议则更多归因于剧本、节奏、角色等基础要素的全面失守。两者虽共享差评中“漫威”、“电影”、“没有”等框架性质疑，但前者更像是高要求未被满足，而后者则是更像未达及格线的批评。

综合上述分析，复联4的情怀属性虽然遭到部分人的质疑，但是仍然打动了大部分观影者，并且其角色塑造深入人心以至于观影者对角色的走向十分关注。雷神虽然受到一定称赞，但其称赞程度与感情显然不及复联4，并且被指剧情无聊，但是其角色塑造与新奇的设定仍为其争取到一定的肯定。

3.2 主题建模

主题建模（Topic Modeling）是一种无监督机器学习方法，用于发现文本文档集合中的隐含主题结构。其核心思想是“文档-主题-词”的三层概率关系。此处使用的模型是LDA（Latent Dirichlet Allocation），即通过隐狄利克雷分布实现建模过程，其生成过程的核心公式为：

文档-主题分布、主题-词分布、词生成过程：

$$P(w|z) = \phi_z^{(w)} \quad (1)$$

$$P(z|d) = \theta_d^{(z)} \quad (2)$$

$$z_{d,n} \sim \text{Multinomial}(\theta_d) w_{d,n} \sim \text{Multinomial}(\phi_{z_{d,n}}) \quad (3)$$

其中 θ_d 表示文档 d 的词分布， ϕ_k 表示主题 k 的词分布， $z_{d,n}$ 是文档 d 中第 n 个词的主题， $w_{d,n}$ 是文档 d 中第 n 个词。

在本研究中，鉴于分开建立主题模型效果较差，此处将所有影评合并后进行主题建模，其结果如下：

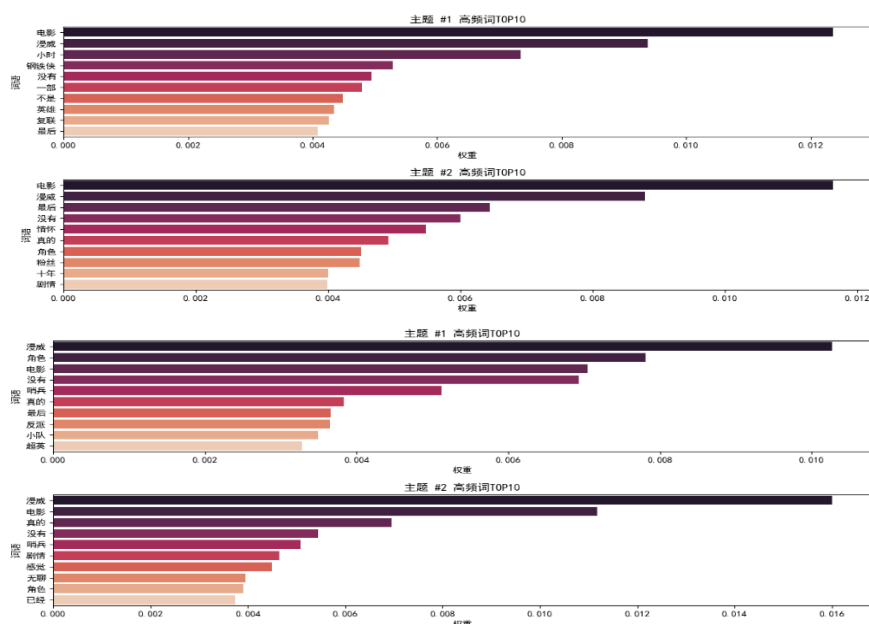


图7 主题建模可视化结果

对于复联 4 的主题 1 关键词集中在“电影”、“漫威”、“小时”等，表明观众对电影整体的体验、片长和漫威 IP 讨论较多；主题 2 关键词包含“漫威”、“剧情”，说明观众关注电影的结局和剧情走向，同时对漫威电影宇宙的整体表现进行了评价。

雷霆特工队的主题 1 包括“角色”、“漫威”，表明观众聚焦于电影中的角色塑造与漫威 IP 的讨论；主题#2 包括“漫威”、“电影”、“真的”、“剧情”、“感觉”，同样指向对电影剧情、真实性和整体感受的关注。与主题 1 相比，主题 2 更加强调主观感受。

复联 4 的讨论聚焦于影片作为漫威品牌巨制的整体体验，其标志性角色（“钢铁侠”）引发的情感共鸣，尤其强调其作为系列终章的宏大叙事与英雄谢幕；而雷霆的讨论则更集中于具体的叙事构建层面，围绕角色塑造（“角色”、“小队”、“反派”）等话题。二者主题建模结果虽共享对漫威、剧情与整体质量的普遍关注，但前者说明观众较关注情怀与后者对角色元素的侧重形成鲜明对比。

3.3 情感分析

情感分析是自然语言处理（NLP）中通过计算手段识别文本主观倾向（如正面/负面/中性）的技术，属于文本分类任务。其实现主要通过构建情感词典（如“好=+1，差=-1”）和语法规则，通过加权计算情感极性：

$$\text{Score} = \sum_{w \in \text{Text}} \text{sentiment}(w) \cdot \text{intensity}(w) \quad (4)$$

其中 w 表示文本中的词语， $\text{sentiment}(w)$ 为情感极性值， $\text{intensity}(w)$ 为强度系数。

本研究借助 Python 中 snownlp 库快速实现中文文本的情感分析，选取 0.4 作为负面评论分界线，选取 0.6 作为正面评论分界线，并计算各情感评论百分比，其结果如下：

表1 情感分析结果表

	复仇者联盟 4	雷霆特工队
总评论数（去空后）	1164	1168
正面评论数	885 (76.0%)	869 (74.4%)
负面评论数	194 (16.7%)	204 (17.5%)
中性评论数	85 (7.3%)	95 (8.1%)

由上表可见。两部电影的影评情感分布均以正面评论为主（两部电影均高于 74%），这表明大部分观影者对这两部电影持乐观态度。数据表明，复联 4 的观众满意度略高于雷特，主要体现在更高正面评论比例（76.0% vs 74.4%）和更低负面比例（16.7% vs 17.5%），但其差异较小。

3.4 基于 LSTM 和 RNN 的文本分类

鉴于豆瓣影评对于单个电影最多只显示 1200 条数据，其近期实际好评率无法根据直接的“好评”计算得到，因此，本研究将实际的“好评”与“差评”作为训练集，将“一般”评论作为测试集，对“一般”评论进行文本二分类为好评与差评，并计算“一般”评论中的好评数，从而得到好评率指标，实现对两部电影口碑的好坏程度进行量化。

LSTM（Long Short-Term Memory）是一种特殊类型的循环神经网络（RNN），专门设计来处理序列数据。其核心优势是解决传统 RNN 的局限性——梯度消失问题，从而能够捕获长期依赖关系。LSTM 通过引入门机制（gates）和细胞状态（cell state）来智能地存储、遗忘和更新信息。主要涉及遗忘门、输入门、输出门三个门机制，核心公式：

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (5)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (6)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (7)$$

最终计算结果如下：

表2 文本分类计算结果

	复仇者联盟 4	雷霆特工队
LSTM 好评率	42.25%	15.75%
LSTM 准确率	0.8565	0.9822
LSTM 验证集准确率	0.7312	0.650
RNN 好评率	60.50%	23.75%
RNN 准确率	0.9786	0.9881
RNN 验证集准确率	0.5125	0.4500

在两个模型的预测中，复仇者联盟 4 的好评率（LSTM 42.25%，RNN 60.50%）均远超雷霆特工队（15.75%与 23.75%），表明粉丝反馈存在明显差异，复联 4 口碑显著优于雷特；模型方面，RNN 系统性地预测更高好评率（在复仇者联盟 4 高 10%、雷霆特工队高 8%），但 LSTM 在验证集上略稳健，RNN 在多次调参后仍存在较大的过拟合问题，而 LSTM 模型的泛化能力更强。

四、 总结

综上所述，各项数据分析结果一致表明，《复仇者联盟 4》在观众口碑上明显胜过《雷霆特工队》。复联 4 凭借其情怀底蕴与史诗般的故事终结，赢得了更广泛的正面评价。相反，雷特虽然在角色塑造上有所亮点，但剧情设计方面的不足导致其负面评价占比相对较高。更进一步的，基于 LSTM 和 RNN 的文本分类模型也印证了这一结论，量化显示复联 4 的好评率显著高于雷特。这项研究结果揭示了情感共鸣和叙事质量在决定观众口碑中的关键作用。本研究为未来电影制作提供了有价值的参考。

参考文献

- [1] Sherstinsky A. Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network[J]. Physica D: Nonlinear Phenomena, 2020, 404: 132306.

附录

代码已同步上传至 github: https://github.com/Yeily/movies_comments_comparement

附录

影评爬取 R 代码:

```
library(rvest)
library(httr)
library(tictoc)
library(tidyverse)

setwd("D:\\R\\rcode\\big_data_cience_work\\project_final_work")
# 数据的爬取
tic()
# 添加请求头(用于随机化)
user_agent <- c(
  "Mozilla/5.0 (Windows NT 5.1; U; en; rv:1.8.1)Gecko/20061208 Firefox/2.0.0
Opera 9.50",
  "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_1)AppleWebKit/537.36
(KHTML, like Gecko) Chrome/71.0.3578.98 Safari/537.36",
  "Mozilla/5.0 (Windows NT 10.0; WOW64; Tri-
dent/7.0; .NET4.0C; .NET4.0E; .NET CLR 2.0.50727; .NET CLR 3.0.30729; .NET CLR
3.5.30729; InfoPath.3; rv:11.0) like Gecko",
  "Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1; Trident/5.0)",
  "Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.0; Trident/4.0)",
  "Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.0)",
  "Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)",
  "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.6; rv:2.0.1) Gecko/20100101 Fire-
fox/4.0.1",
  "Mozilla/5.0 (Windows NT 6.1; rv:2.0.1) Gecko/20100101 Firefox/4.0.1",
  "Opera/9.80 (Macintosh; Intel Mac OS X 10.6.8; U; en) Presto/2.8.131 Ver-
sion/11.11",
  "Opera/9.80 (Windows NT 6.1; U; en) Presto/2.8.131 Version/11.11",
  "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_7_0) AppleWebKit/535.11 (KHTML,
like Gecko) Chrome/17.0.963.56 Safari/535.11",
  "Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1; Maxthon 2.0)",
  "Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1; TencentTraveler 4.0)",
  "Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1)",
  "Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1; The World)",
  "Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1; Trident/4.0; SE 2.X MetaSr
1.0; SE 2.X MetaSr 1.0; .NET CLR 2.0.50727; SE 2.X MetaSr 1.0)",
  "Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1; 360SE)",
  "Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1; Avant Browser)",
  "Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1)"
)
cookie <- read_file("cookie.txt") # 此处为保护隐私使用了读取的方式, 复现时
若删去 cookie 则评论数量会有更大的限制
headers <- add_headers(
  'User-Agent' = sample(user_agent,1),
```

```

'Referer' = "https://movie.douban.com/",
'Accept-Language' = "zh-CN,zh;q=0.9,en;q=0.8",
'Cookie' = cookie
)
# 复仇者联盟 4
# 复仇者联盟 4 好评
url1_high <- paste("https://movie.douban.com/subject/26100958/comments?percent_type=h&start=", (0:19)*20, "&limit=20&status=P&sort=new_score", sep="")
text1_h <- c()
for (i in url1_high){
  print(i)
  response <- GET(i, headers)
  page <- read_html(content(response, "text", encoding = "utf-8"))
  comment <- page %>%
    html_nodes(".comment-content span") %>%
    html_text(trim = TRUE) %>%
    str_replace_all("[[:punct:]]|[\n]", "") %>%
    str_replace_all("[\U0001F600-\U0001F64F]", "") %>%
    str_replace_all("[\U0001F300-\U0001F5FF]", "") %>%
    str_replace_all("[\U0001F680-\U0001F6FF]", "") %>%
    str_replace_all("[\U0001F1E0-\U0001F1FF]", "") %>%
    str_replace_all("[\U00002695-\U0000269F]", "") %>%
    str_replace_all("[\U00002600-\U00002B55]", "") %>%
    str_replace_all("[\U0000231A-\U0000231B]", "") %>%
    str_replace_all("[\U000025FB-\U000025FE]", "") %>%
    str_replace_all("[\U00002000-\U0000206F]", "")
  text1_h <- c(text1_h, comment)
  Sys.sleep(runif(1, 0.5, 1)) # 随机延时 0.5 秒到 1 秒之间
}
# 复仇者联盟 4 差评
text1_l <- c()
url1_low <- paste("https://movie.douban.com/subject/26100958/comments?percent_type=l&start=", (0:19)*20, "&limit=20&status=P&sort=new_score", sep="")
for (i in url1_low){
  print(i)
  response <- GET(i, headers)
  page <- read_html(content(response, "text", encoding = "utf-8"))
  comment <- page %>%
    html_nodes(".comment-content span") %>%
    html_text(trim = TRUE) %>%
    str_replace_all("[[:punct:]]|[\n]", "") %>%
    str_replace_all("[\U0001F600-\U0001F64F]", "") %>%
    str_replace_all("[\U0001F300-\U0001F5FF]", "") %>%
    str_replace_all("[\U0001F680-\U0001F6FF]", "") %>%
    str_replace_all("[\U0001F1E0-\U0001F1FF]", "") %>%
    str_replace_all("[\U00002695-\U0000269F]", "") %>%
    str_replace_all("[\U00002600-\U00002B55]", "") %>%
    str_replace_all("[\U0000231A-\U0000231B]", "") %>%
    str_replace_all("[\U000025FB-\U000025FE]", "") %>%
    str_replace_all("[\U00002000-\U0000206F]", "")

```

```

    text1_1 <- c(text1_1, comment)
    Sys.sleep(runif(1, 0.5, 1))
  }
  # 复仇者联盟 4 一般评论
  text1_m <- c()
  url1_median <- paste("https://movie.douban.com/subject/26100958/comments?per-
cent_type=m&start=", (0:19)*20, "&limit=20&status=P&sort=new_score", sep="")
  for (i in url1_median){
    print(i)
    response <- GET(i, headers)
    page <- read_html(content(response, "text", encoding = "utf-8"))
    comment <- page %>%
      html_nodes(".comment-content span") %>%
      html_text(trim = TRUE) %>%
      str_replace_all("[[:punct:]]|[\n]", "") %>%
      str_replace_all("[\U0001F600-\U0001F64F]", "") %>%
      str_replace_all("[\U0001F300-\U0001F5FF]", "") %>%
      str_replace_all("[\U0001F680-\U0001F6FF]", "") %>%
      str_replace_all("[\U0001F1E0-\U0001F1FF]", "") %>%
      str_replace_all("[\U00002695-\U0000269F]", "") %>%
      str_replace_all("[\U00002600-\U00002B55]", "") %>%
      str_replace_all("[\U0000231A-\U0000231B]", "") %>%
      str_replace_all("[\U000025FB-\U000025FE]", "") %>%
      str_replace_all("[\U00002000-\U0000206F]", "")
    text1_m <- c(text1_m, comment)
    Sys.sleep(runif(1, 0.5, 1))
  }
  # 雷霆特工队
  # 雷霆特工队好评
  text2_h <- c()
  url2_high <- paste("https://movie.douban.com/subject/35927475/comments?per-
cent_type=h&start=", (0:19)*20, "&limit=20&status=P&sort=new_score", sep="")
  for (i in url2_high){
    print(i)
    response <- GET(i, headers)
    page <- read_html(content(response, "text", encoding = "utf-8"))
    comment <- page %>%
      html_nodes(".comment-content span") %>%
      html_text(trim = TRUE) %>%
      str_replace_all("[[:punct:]]|[\n]", "") %>%
      str_replace_all("[\U0001F600-\U0001F64F]", "") %>%
      str_replace_all("[\U0001F300-\U0001F5FF]", "") %>%
      str_replace_all("[\U0001F680-\U0001F6FF]", "") %>%
      str_replace_all("[\U0001F1E0-\U0001F1FF]", "") %>%
      str_replace_all("[\U00002695-\U0000269F]", "") %>%
      str_replace_all("[\U00002600-\U00002B55]", "") %>%
      str_replace_all("[\U0000231A-\U0000231B]", "") %>%
      str_replace_all("[\U000025FB-\U000025FE]", "") %>%
      str_replace_all("[\U00002000-\U0000206F]", "")
    text2_h <- c(text2_h, comment)
  }

```

```

    Sys.sleep(runif(1, 0.5, 1))
  }
  # 雷霆特工队差评
  text2_l <- c()
  url2_low <- paste("https://movie.douban.com/subject/35927475/comments?percent_type=l&start=", (0:19)*20, "&limit=20&status=P&sort=new_score", sep="")
  for (i in url2_low){
    print(i)
    response <- GET(i, headers)
    page <- read_html(content(response, "text", encoding = "utf-8"))
    comment <- page %>%
      html_nodes(".comment-content span") %>%
      html_text(trim = TRUE) %>%
      str_replace_all("[[:punct:]]|[\n]", "") %>%
      str_replace_all("[\U0001F600-\U0001F64F]", "") %>%
      str_replace_all("[\U0001F300-\U0001F5FF]", "") %>%
      str_replace_all("[\U0001F680-\U0001F6FF]", "") %>%
      str_replace_all("[\U0001F1E0-\U0001F1FF]", "") %>%
      str_replace_all("[\U00002695-\U0000269F]", "") %>%
      str_replace_all("[\U00002600-\U00002B55]", "") %>%
      str_replace_all("[\U0000231A-\U0000231B]", "") %>%
      str_replace_all("[\U000025FB-\U000025FE]", "") %>%
      str_replace_all("[\U00002000-\U0000206F]", "")
    text2_l <- c(text2_l, comment)
    Sys.sleep(runif(1, 0.5, 1))
  }
  # 雷霆特工队一般评论
  text2_m <- c()
  url2_median <- paste("https://movie.douban.com/subject/35927475/comments?percent_type=l&start=", (0:19)*20, "&limit=20&status=P&sort=new_score", sep="")
  for (i in url2_median){
    print(i)
    response <- GET(i, headers)
    page <- read_html(content(response, "text", encoding = "utf-8"))
    comment <- page %>%
      html_nodes(".comment-content span") %>%
      html_text(trim = TRUE) %>%
      str_replace_all("[[:punct:]]|[\n]", "") %>%
      str_replace_all("[\U0001F600-\U0001F64F]", "") %>%
      str_replace_all("[\U0001F300-\U0001F5FF]", "") %>%
      str_replace_all("[\U0001F680-\U0001F6FF]", "") %>%
      str_replace_all("[\U0001F1E0-\U0001F1FF]", "") %>%
      str_replace_all("[\U00002695-\U0000269F]", "") %>%
      str_replace_all("[\U00002600-\U00002B55]", "") %>%
      str_replace_all("[\U0000231A-\U0000231B]", "") %>%
      str_replace_all("[\U000025FB-\U000025FE]", "") %>%
      str_replace_all("[\U00002000-\U0000206F]", "")
    text2_m <- c(text2_m, comment)
  }
  toc()

```

```

# 创建数据框
comments1 <- data.frame(
  Comment = c(text1_h, text1_l, text1_m),
  Type = c(rep("好评", length(text1_h)), rep("差评", length(text1_l)), rep("一般",
length(text1_m))),
  stringsAsFactors = FALSE
)
# 创建数据框
comments2 <- data.frame(
  Comment = c(text2_h, text2_l, text2_m),
  Type = c(rep("好评", length(text1_h)), rep("差评", length(text1_l)), rep("一般",
length(text1_m))),
  stringsAsFactors = FALSE
)
# 保存到 CSV 文件
write.csv(comments1, "Avengers-Endgame_comments.csv", row.names = FALSE,
fileEncoding = "utf-8")
write.csv(comments2, "Thunderbolts_comments.csv", row.names = FALSE, fileEn-
coding = "utf-8")

```

词云图及主题建模 py 代码:

```

from data_handling import DoubanTextCleaner
from wordcloud import WordCloud
import matplotlib.pyplot as plt
from gensim import corpora, models
import pandas as pd
import pyLDAvis.gensim_models
import seaborn as sns
from collections import Counter
import numpy as np
import os

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# ===== 可视化模块增强版 =====
class EnhancedVisualizer:
    @staticmethod
    def generate_wordcloud(words, title, font_path='simhei.ttf'):
        """生成中文词云图"""
        word_freq = dict(Counter(words).most_common(200))

        wc = WordCloud(
            font_path=font_path,
            width=800,
            height=600,
            background_color='white',
            max_words=200,

```

```

        max_font_size=100
    ).generate_from_frequencies(word_freq)

    plt.figure(figsize=(10, 8))
    plt.imshow(wc, interpolation='bilinear')
    plt.title(f'{title} - 词云图', fontsize=16)
    plt.axis('off')
    plt.show()
    return word_freq  # 返回词频用于进一步分析

    @staticmethod
    def plot_word_frequency(word_freq, title, top_n=10):
        """绘制词频柱状图"""
        top_words = dict(sorted(word_freq.items(), key=lambda x: x[1], reverse=True)[:top_n])

        plt.figure(figsize=(12, 8))
        sns.barplot(
            x=list(top_words.values()),
            y=list(top_words.keys()),
            palette="viridis"
        )
        plt.title(f'{title} - 高频词 TOP{top_n}', fontsize=14)
        plt.xlabel("出现频次")
        plt.ylabel("词语")
        plt.tight_layout()
        plt.show()

    @staticmethod
    def visualize_lda(lda_model, corpus, dictionary, html_path="lda_visualization.html"):
        """主题模型可视化(pyLDAvis)"""
        vis = pyLDAvis.gensim_models.prepare(
            lda_model,
            corpus,
            dictionary,
            sort_topics=False
        )
        # 保存为 HTML 文件并自动打开
        pyLDAvis.save_html(vis, html_path)
        os.system(f'start {html_path}')  # Windows 系统自动打开
        return vis

    @staticmethod
    def plot_topic_words(lda_model, num_topics=3, num_words=10):
        """绘制每个主题的前 N 个高频词柱形图"""
        topics = lda_model.show_topics(num_topics=num_topics, num_words=num_words, formatted=False)

```

```

plt.figure(figsize=(12, 8))
for topic_id, topic_words in topics:
    # 提取词和权重
    words = [word for word, _ in topic_words]
    weights = [weight for _, weight in topic_words]

    # 绘制柱形图
    plt.subplot(num_topics, 1, topic_id+1)
    sns.barplot(x=weights, y=words, palette="rocket")
    plt.title(f'主题 #{topic_id+1} 高频词 TOP{num_words}', font-
size=12)

    plt.xlabel("权重")
    plt.ylabel("词语")

plt.tight_layout()
plt.show()

# ===== 主题建模模块 =====
class TopicModeler:
    @staticmethod
    def lda_model(texts, num_topics=3, passes=15):
        """LDA 主题建模"""
        dictionary = corpora.Dictionary([text.split() for text in texts])
        corpus = [dictionary.doc2bow(text.split()) for text in texts]

        lda = models.LdaModel(
            corpus=corpus,
            id2word=dictionary,
            num_topics=num_topics,
            passes=passes,
            random_state=1
        )

        return lda, corpus, dictionary

    @staticmethod
    def show_topics(lda_model, num_words=10):
        """显示每个主题的关键词"""
        return lda_model.print_topics(num_words=num_words)

# ===== 主业务流程 =====
def task1():
    """任务 1：生成复仇者联盟 4 和雷霆特工队的评论词云和主题建模
    读取两部电影的评论数据，生成词云、词频分析，并进行主题建模。
    """
    # 初始化文本处理器
    cleaner = DoubanTextCleaner()

    # 读取数据

```

```

data1 = pd.read_csv('Avengers-Endgame_comments.csv').to_dict('records')
data2 = pd.read_csv('Thunderbolts_comments.csv').to_dict('records')

# 生成词云和词频分析
def process_comments(data, title):
    # 词云和词频统计
    for comment_type in ['好评', '差评', '一般']:
        print(f"\n 正在处理 《{title}》 - {comment_type}...")
        words = cleaner.preprocess_for_wordcloud(
            texts=data,
            type_filter=comment_type
        )

        # 词云可视化
        word_freq = EnhancedVisualizer.generate_wordcloud(
            words,
            f'{title} - {comment_type}'
        )

        # 词频柱状图
        if word_freq: # 确保有数据才绘制
            EnhancedVisualizer.plot_word_frequency(
                word_freq,
                f'{title} - {comment_type}'
            )

# 处理两部电影数据
process_comments(data1, "复仇者联盟 4")
process_comments(data2, "雷霆特工队")

# 主题建模
print("\n 正在进行主题建模...")
segmented_texts1 = [
    cleaner.preprocess_for_analysis(str(item['Comment']))
    for item in data1
]
segmented_texts2 = [
    cleaner.preprocess_for_analysis(str(item['Comment']))
    for item in data2
]

# 运行 LDA 模型
lda, corpus, dictionary = TopicModeler.lda_model(segmented_texts1, num_topics=2)
lda2, corpus2, dictionary2 = TopicModeler.lda_model(segmented_texts2, num_topics=2)

# 显示主题关键词

```

```

topics = TopicModeler.show_topics(lda)
topics2 = TopicModeler.show_topics(lda2)
print("\n 主题关键词分布:")
for idx, topic in topics:
    print(f"\n 主题 #{idx+1}:")
    print(topic)

for idx, topic in topics2:
    print(f"\n 主题 #{idx+1}:")
    print(topic)

# 主题模型可视化
print("\n 生成主题模型可视化(浏览器自动打开)...")
EnhancedVisualizer.visualize_lda(lda, corpus, dictionary, "lda_visualiza-
tion_avengers.html")
EnhancedVisualizer.visualize_lda(lda2, corpus2, dictionary2, "lda_visualiza-
tion_thunderbolts.html")

# 主题词柱形图可视化
print("\n 生成主题高频词柱形图...")
EnhancedVisualizer.plot_topic_words(lda)
EnhancedVisualizer.plot_topic_words(lda2)

if __name__ == "__main__":
    task1()

```

情感分析 py 代码:

```

import pandas as pd
from snownlp import SnowNLP
from data_handling import DoubanTextCleaner

def analyze_sentiment(df):
    # 初始化文本清洗器
    cleaner = DoubanTextCleaner()

    # 情感分析结果存储
    results = {
        'positive': 0,
        'negative': 0,
        'neutral': 0,
        'total': 0
    }

    # 分析每条评论
    for comment in df['Comment'].dropna():
        try:
            # 预处理文本
            processed = cleaner.preprocess_for_analysis(comment)

```

```

        # 跳过空文本
        if not processed.strip():
            continue

        # 情感分析
        s = SnowNLP(processed)
        sentiment = s.sentiments

        # 分类情感
        if sentiment > 0.6:
            results['positive'] += 1
        elif sentiment < 0.4:
            results['negative'] += 1
        else:
            results['neutral'] += 1
        results['total'] += 1

    except Exception as e:
        print(f'分析评论时出错: {comment[:50]}... 错误: {str(e)}')

# 计算百分比
if results['total'] > 0:
    results['positive_pct'] = results['positive'] / results['total'] * 100
    results['negative_pct'] = results['negative'] / results['total'] * 100
    results['neutral_pct'] = results['neutral'] / results['total'] * 100
else:
    results['positive_pct'] = 0
    results['negative_pct'] = 0
    results['neutral_pct'] = 0

return results

def task2():
    """
    任务 2: 情感分析
    读取复仇者联盟 4 和雷霆特工队的评论数据，进行情感分析。
    """
    # 读取 CSV 文件
    df_avengers = pd.read_csv("Avengers-Endgame_comments.csv")
    df_thunderbolts = pd.read_csv("Thunderbolts_comments.csv")

    # 分析复仇者联盟数据
    print("=== 复仇者联盟 4:终局之战 ===")
    avengers_analysis = analyze_sentiment(df_avengers)
    print(f'总评论数: {avengers_analysis["total"]}')
    print(f'正面评论: {avengers_analysis["positive"]} ({avengers_analysis["positive_pct"]:.1f}%)')
    print(f'负面评论: {avengers_analysis["negative"]} ({avengers_analysis["negative_pct"]:.1f}%)')

```

```

        print(f' 中性评论: {avengers_analysis['neutral']} ({avengers_analysis['neutral_pct']:.1f}%)')

    # 分析雷霆特工队数据
    print("\n=== 雷霆特工队 ===")
    thunderbolts_analysis = analyze_sentiment(df_thunderbolts)
    print(f'总评论数: {thunderbolts_analysis['total']}')
    print(f' 正面评论: {thunderbolts_analysis['positive']} ({thunderbolts_analysis['positive_pct']:.1f}%)')
    print(f' 负面评论: {thunderbolts_analysis['negative']} ({thunderbolts_analysis['negative_pct']:.1f}%)')
    print(f' 中性评论: {thunderbolts_analysis['neutral']} ({thunderbolts_analysis['neutral_pct']:.1f}%)')

if __name__ == "__main__":
    task2()

```

复联 4LSTM 的 py 代码:

```

import re
import pandas as pd
import numpy as np
import jieba
import matplotlib.pyplot as plt
from data_handling import DoubanTextCleaner
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.utils import to_categorical

def load_and_preprocess_data(input_csv, comment_col='Comment',
                             type_col='Type'):
    """加载数据并进行预处理"""
    df = pd.read_csv(input_csv)
    comments = df[comment_col].values
    types = df[type_col].values

    text_cleaner = DoubanTextCleaner()
    comments = [comment if pd.notna(comment) else " for comment in comments]
    comments = [text_cleaner.preprocess_for_analysis(str(comment)) for comment
in comments]
    return df, np.array(comments), types

def prepare_datasets(comments, types):
    """准备训练和预测数据集"""
    train_mask = np.isin(types, ['好评', '差评'])

```

```

predict_mask = types == '一般'

X_train = comments[train_mask]
y_train = types[train_mask]
X_predict = comments[predict_mask]

label_map = {'差评': 0, '好评': 1}
y_train = np.array([label_map[label] for label in y_train])
y_train = to_categorical(y_train)

return X_train, y_train, X_predict, predict_mask

def build_lstm_model(vocab_size=5000, max_len=200):
    """构建 LSTM 模型"""
    model = Sequential()
    model.add(Embedding(input_dim=vocab_size, output_dim=256, input_length=max_len))
    model.add(LSTM(8, dropout=0.2, recurrent_dropout=0.2))
    model.add(Dense(2, activation='relu'))
    model.add(Dropout(0.6))
    model.add(Dense(2, activation='softmax'))
    model.compile(loss='categorical_crossentropy',
                  optimizer='adamw',
                  metrics=['accuracy'])
    return model

def train_and_evaluate(model, X_train_pad, y_train, epochs=50, batch_size=64):
    """训练模型并评估"""
    history = model.fit(X_train_pad, y_train,
                        batch_size=batch_size,
                        epochs=epochs,
                        validation_split=0.2)
    return history

def visualize_training(history, output_img):
    """可视化训练过程"""
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='训练准确率')
    plt.plot(history.history['val_accuracy'], label='验证准确率')
    plt.title('模型准确率')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'], label='训练损失')
    plt.plot(history.history['val_loss'], label='验证损失')
    plt.title('模型损失')

```

```

plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.savefig(output_img)
plt.show()

def predict_and_save(model, tokenizer, df, X_predict, predict_mask, output_csv):
    """预测并保存结果"""
    X_predict_seq = tokenizer.texts_to_sequences(X_predict)
    X_predict_pad = pad_sequences(X_predict_seq, maxlen=200)
    predictions = model.predict(X_predict_pad)
    predicted_labels = ['好评' if pred[1] > 0.5 else '差评' for pred in predictions]

    df.loc[predict_mask, 'Predicted_Type'] = predicted_labels
    df.to_csv(output_csv, index=False)

    general_comments = df[df['Type'] == '一般']
    good_ratio = len(general_comments[general_comments['Predicted_Type'] == '好评']) / len(general_comments)
    print(f"\n 一般评论中好评占比: {good_ratio:.2%}")
    print(f"\n 分类完成， 结果已保存到{output_csv}")

def run_lstm_analysis(input_csv, output_csv, metrics_img, comment_col='Comment', type_col='Type'):
    """主函数： 运行完整的 LSTM 分析流程"""
    # 1. 数据加载和预处理
    df, comments, types = load_and_preprocess_data(input_csv, comment_col, type_col)

    # 2. 准备数据集
    X_train, y_train, X_predict, predict_mask = prepare_datasets(comments, types)

    # 3. 文本向量化
    tokenizer = Tokenizer(num_words=5000)
    tokenizer.fit_on_texts(X_train)
    X_train_seq = tokenizer.texts_to_sequences(X_train)
    X_train_pad = pad_sequences(X_train_seq, maxlen=200)

    # 4. 构建和训练模型
    model = build_lstm_model()
    history = train_and_evaluate(model, X_train_pad, y_train)

    # 5. 可视化训练过程
    visualize_training(history, metrics_img)

    # 6. 预测并保存结果
    predict_and_save(model, tokenizer, df, X_predict, predict_mask, output_csv)

```

```

def task3():
    """
    任务 3: LSTM 文本分类
    读取复联 4 的评论数据, 进行 LSTM 文本分类。
    """
    # 分析复仇者联盟 4 评论
    run_lstm_analysis(
        input_csv='Avengers-Endgame_comments.csv',
        output_csv='av_classified_comments.csv',
        metrics_img='av_training_metrics_test.png'
    )

if __name__ == "__main__":
    task3()
    print("LSTM 文本分类任务已完成。")

```

雷霆特工队 LSTM 的 py 代码:

```

import re
import pandas as pd
import numpy as np
import jieba
import matplotlib.pyplot as plt
from data_handling import DoubanTextCleaner
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.utils import to_categorical

def load_and_preprocess_data(input_csv, comment_col='Comment',
                             type_col='Type'):
    """加载数据并进行预处理"""
    df = pd.read_csv(input_csv)
    comments = df[comment_col].values
    types = df[type_col].values

    text_cleaner = DoubanTextCleaner()
    comments = [comment if pd.notna(comment) else " for comment in comments]
    comments = [text_cleaner.preprocess_for_analysis(str(comment)) for comment
in comments]
    return df, np.array(comments), types

def prepare_datasets(comments, types):
    """准备训练和预测数据集"""
    train_mask = np.isin(types, ['好评', '差评'])

```

```

predict_mask = types == '一般'

X_train = comments[train_mask]
y_train = types[train_mask]
X_predict = comments[predict_mask]

label_map = {'差评': 0, '好评': 1}
y_train = np.array([label_map[label] for label in y_train])
y_train = to_categorical(y_train)

return X_train, y_train, X_predict, predict_mask

def build_lstm_model(vocab_size=5000, max_len=200):
    """构建 LSTM 模型"""
    model = Sequential()
    model.add(Embedding(input_dim=vocab_size, output_dim=128, input_length=max_len))
    model.add(LSTM(128, dropout=0.4, recurrent_dropout=0.4))
    model.add(Dropout(0.2))
    model.add(Dense(2, activation='softmax'))
    model.compile(loss='categorical_crossentropy',
                  optimizer='adamw',
                  metrics=['accuracy'])
    return model

def train_and_evaluate(model, X_train_pad, y_train, epochs=50, batch_size=64):
    """训练模型并评估"""
    history = model.fit(X_train_pad, y_train,
                        batch_size=batch_size,
                        epochs=epochs,
                        validation_split=0.2)
    return history

def visualize_training(history, output_img):
    """可视化训练过程"""
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='训练准确率')
    plt.plot(history.history['val_accuracy'], label='验证准确率')
    plt.title('模型准确率')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'], label='训练损失')
    plt.plot(history.history['val_loss'], label='验证损失')
    plt.title('模型损失')
    plt.xlabel('Epoch')

```

```

plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.savefig(output_img)
plt.show()

def predict_and_save(model, tokenizer, df, X_predict, predict_mask, output_csv):
    """预测并保存结果"""
    X_predict_seq = tokenizer.texts_to_sequences(X_predict)
    X_predict_pad = pad_sequences(X_predict_seq, maxlen=200)
    predictions = model.predict(X_predict_pad)
    predicted_labels = ['好评' if pred[1] > 0.5 else '差评' for pred in predictions]

    df.loc[predict_mask, 'Predicted_Type'] = predicted_labels
    df.to_csv(output_csv, index=False)

    general_comments = df[df['Type'] == '一般']
    good_ratio = len(general_comments[general_comments['Predicted_Type'] == '好评']) / len(general_comments)
    print(f"\n 一般评论中好评占比: {good_ratio:.2%}")
    print(f"\n 分类完成， 结果已保存到{output_csv}")

def run_lstm_analysis(input_csv, output_csv, metrics_img, comment_col='Comment', type_col='Type'):
    """主函数： 运行完整的 LSTM 分析流程"""
    # 1. 数据加载和预处理
    df, comments, types = load_and_preprocess_data(input_csv, comment_col, type_col)

    # 2. 准备数据集
    X_train, y_train, X_predict, predict_mask = prepare_datasets(comments, types)

    # 3. 文本向量化
    tokenizer = Tokenizer(num_words=5000)
    tokenizer.fit_on_texts(X_train)
    X_train_seq = tokenizer.texts_to_sequences(X_train)
    X_train_pad = pad_sequences(X_train_seq, maxlen=200)

    # 4. 构建和训练模型
    model = build_lstm_model()
    history = train_and_evaluate(model, X_train_pad, y_train)

    # 5. 可视化训练过程
    visualize_training(history, metrics_img)

    # 6. 预测并保存结果
    predict_and_save(model, tokenizer, df, X_predict, predict_mask, output_csv)

```

```

def task4():
    """
    任务 4: LSTM 文本分类
    读取雷霆特工队的评论数据，进行 LSTM 文本分类。
    """

    # 分析评论
    results = run_lstm_analysis(
        input_csv='Thunderbolts_comments.csv',
        output_csv='th_classified_comments.csv',
        metrics_img='th_training_metrics_test.png'
    )

    print(f'情感分析结果: {results}')

if __name__ == "__main__":
    task4()
    print("LSTM 文本分类任务已完成。")

```

RNN 的 py 代码:

```

import re
import pandas as pd
import numpy as np
import jieba
import matplotlib.pyplot as plt
from data_handling import DoubanTextCleaner
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense, Dropout
from tensorflow.keras.utils import to_categorical

def load_and_preprocess_data(input_csv, comment_col='Comment',
                             type_col='Type'):
    """加载数据并进行预处理"""
    df = pd.read_csv(input_csv)
    comments = df[comment_col].values
    types = df[type_col].values

    text_cleaner = DoubanTextCleaner()
    comments = [comment if pd.notna(comment) else " for comment in comments]
    comments = [text_cleaner.preprocess_for_analysis(str(comment)) for comment
in comments]
    return df, np.array(comments), types

def prepare_datasets(comments, types):

```

```

"""准备训练和预测数据集"""
train_mask = np.isin(types, ['好评', '差评'])
predict_mask = types == '一般'

X_train = comments[train_mask]
y_train = types[train_mask]
X_predict = comments[predict_mask]

label_map = {'差评': 0, '好评': 1}
y_train = np.array([label_map[label] for label in y_train])
y_train = to_categorical(y_train)

return X_train, y_train, X_predict, predict_mask

def build_rnn_model(vocab_size=5000, max_len=100):
    """构建 RNN 模型"""
    model = Sequential()
    model.add(Embedding(input_dim=vocab_size, output_dim=32, input_length=max_len))
    model.add(SimpleRNN(8, dropout=0.5))
    model.add(Dropout(0.5))
    model.add(Dense(2, activation='softmax'))
    model.compile(loss='categorical_crossentropy',
                  optimizer='adamw',
                  metrics=['accuracy'])
    return model

def train_and_evaluate(model, X_train_pad, y_train, epochs=50, batch_size=32):
    """训练模型并评估"""
    history = model.fit(X_train_pad, y_train,
                       batch_size=batch_size,
                       epochs=epochs,
                       validation_split=0.2)

    return history

def visualize_training(history, output_img):
    """可视化训练过程"""
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='训练准确率')
    plt.plot(history.history['val_accuracy'], label='验证准确率')
    plt.title('模型准确率')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'], label='训练损失')
    plt.plot(history.history['val_loss'], label='验证损失')

```

```

plt.title('模型损失')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.savefig(output_img)
plt.show()

def predict_and_save(model, tokenizer, df, X_predict, predict_mask, output_csv):
    """预测并保存结果"""
    X_predict_seq = tokenizer.texts_to_sequences(X_predict)
    X_predict_pad = pad_sequences(X_predict_seq, maxlen=200)
    predictions = model.predict(X_predict_pad)
    predicted_labels = ['好评' if pred[1] > 0.5 else '差评' for pred in predictions]

    df.loc[predict_mask, 'Predicted_Type'] = predicted_labels
    df.to_csv(output_csv, index=False)

    general_comments = df[df['Type'] == '一般']
    good_ratio = len(general_comments[general_comments['Predicted_Type'] == '好评']) / len(general_comments)
    print(f'\n 一般评论中好评占比: {good_ratio:.2%}')
    print(f'\n 分类完成, 结果已保存到{output_csv}')

def run_rnn_analysis(input_csv, output_csv, metrics_img, comment_col='Comment',
type_col='Type'):
    """主函数: 运行完整的 RNN 分析流程"""
    # 1. 数据加载和预处理
    df, comments, types = load_and_preprocess_data(input_csv, comment_col,
type_col)

    # 2. 准备数据集
    X_train, y_train, X_predict, predict_mask = prepare_datasets(comments, types)

    # 3. 文本向量化
    tokenizer = Tokenizer(num_words=5000)
    tokenizer.fit_on_texts(X_train)
    X_train_seq = tokenizer.texts_to_sequences(X_train)
    X_train_pad = pad_sequences(X_train_seq, maxlen=100)

    # 4. 构建和训练模型
    model = build_rnn_model()
    history = train_and_evaluate(model, X_train_pad, y_train)

    # 5. 可视化训练过程
    visualize_training(history, metrics_img)

```

```

# 6. 预测并保存结果
predict_and_save(model, tokenizer, df, X_predict, predict_mask, output_csv)

def task5():
    """
    任务 5: RNN 文本分类
    读取复联 4 和雷霆特工队的评论数据, 进行 RNN 文本分类。
    """
    # 分析复仇者联盟评论
    print("正在分析复仇者联盟评论...")
    run_rnn_analysis(
        input_csv='Avengers-Endgame_comments.csv',
        output_csv='av_classified_comments_rnn.csv',
        metrics_img='av_training_metrics_rnn.png'
    )

    # 分析雷霆特工队评论
    print("\n 正在分析雷霆特工队评论...")
    run_rnn_analysis(
        input_csv='Thunderbolts_comments.csv',
        output_csv='th_classified_comments_rnn.csv',
        metrics_img='th_training_metrics_rnn.png'
    )

if __name__ == "__main__":
    task5()
    print("RNN 文本分类任务已完成。")

```

主程序:

```

import os
from data_handling import DoubanTextCleaner
from wordcloudTopsis import task1 as wordcloud_task
from sentiment_analysis import task2 as sentiment_task
from lstm_comment_av import task3 as lstm_av_task
from lstm_comment_th import task4 as lstm_th_task
from rnn_comment import task5 as rnn_task

def main():
    """主程序: 整合所有分析模块"""
    print("=== 影评分析系统 ===")

    # 1. 初始化文本处理器
    print("\n 初始化文本处理器...")
    cleaner = DoubanTextCleaner()

    # 2. 词云和主题分析
    print("\n=== 开始词云和主题分析 ===")
    wordcloud_task()

```

```
# 3. 情感分析
print("\n=== 开始情感分析 ===")
sentiment_task()

# 4. LSTM 分类
print("\n=== 开始 LSTM 分类 ===")
print("\n 分析复仇者联盟评论...")
lstm_av_task()
print("\n 分析雷霆特工队评论...")
lstm_th_task()

# 5. RNN 分类
print("\n=== 开始 RNN 分类 ===")
rnn_task()

print("\n=== 所有分析任务已完成 ===")
print("结果文件已保存在当前目录:")
print("- av_classified_comments.csv: 复仇者联盟 LSTM 分类结果")
print("- th_classified_comments.csv: 雷霆特工队 LSTM 分类结果")
print("- av_classified_comments_rnn.csv: 复仇者联盟 RNN 分类结果")
print("- th_classified_comments_rnn.csv: 雷霆特工队 RNN 分类结果")
print("- lda_visualization.html: 主题模型可视化")
print("- *.png: 各种分析图表")

if __name__ == "__main__":
    main()
```